



Animations

A set of animation that illustrates concepts from this chapter is available at the Premium Web site. The reader is encouraged to view the animations to reinforce concepts from this chapter.

3.9 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

blocked state child process exit state interrupt kernel mode mode switch new state parent process preempt	privileged mode process process control block process image process switch program status word ready state round robin running state	suspend state swapping system mode task trace trap user mode
---	--	--

Review Questions

- 3.1 What is an instruction trace?
- 3.2 What common events lead to the creation of a process?
- 3.3 For the processing model of Figure 3.6, briefly define each state.
- 3.4 What does it mean to preempt a process?
- 3.5 What is swapping and what is its purpose?
- 3.6 Why does Figure 3.9b have two blocked states?
- 3.7 List four characteristics of a suspended process.
- 3.8 For what types of entities does the OS maintain tables of information for management purposes?
- 3.9 List three general categories of information in a process control block.
- 3.10 Why are two modes (user and kernel) needed?
- 3.11 What are the steps performed by an OS to create a new process?
- 3.12 What is the difference between an interrupt and a trap?
- 3.13 Give three examples of an interrupt.
- 3.14 What is the difference between a mode switch and a process switch?

Problems

- 3.1 The following state transition table is a simplified model of process management, with the labels representing transitions between states of READY, RUN, BLOCKED, and NONRESIDENT.

	READY	RUN	BLOCKED	NONRESIDENT
READY	–	1	–	5
RUN	2	–	3	–
BLOCKED	4	–	–	6

Give an example of an event that can cause each of the above transitions. Draw a diagram if that helps.

- 3.2** Assume that at time 5 no system resources are being used except for the processor and memory. Now consider the following events:

At time 5: P1 executes a command to read from disk unit 3.
 At time 15: P5's time slice expires.
 At time 18: P7 executes a command to write to disk unit 3.
 At time 20: P3 executes a command to read from disk unit 2.
 At time 24: P5 executes a command to write to disk unit 3.
 At time 28: P5 is swapped out.
 At time 33: An interrupt occurs from disk unit 2: P3's read is complete.
 At time 36: An interrupt occurs from disk unit 3: P1's read is complete.
 At time 38: P8 terminates.
 At time 40: An interrupt occurs from disk unit 3: P5's write is complete.
 At time 44: P5 is swapped back in.
 At time 48: An interrupt occurs from disk unit 3: P7's write is complete.

For each time 22, 37, and 47, identify which state each process is in. If a process is blocked, further identify the event on which it is blocked.

- 3.3** Figure 3.9b contains seven states. In principle, one could draw a transition between any two states, for a total of 42 different transitions.

- List all of the possible transitions and give an example of what could cause each transition.
- List all of the impossible transitions and explain why.

- 3.4** For the seven-state process model of Figure 3.9b, draw a queueing diagram similar to that of Figure 3.8b.

- 3.5** Consider the state transition diagram of Figure 3.9b. Suppose that it is time for the OS to dispatch a process and that there are processes in both the Ready state and the Ready/Suspend state, and that at least one process in the Ready/Suspend state has higher scheduling priority than any of the processes in the Ready state. Two extreme policies are as follows: (1) Always dispatch from a process in the Ready state, to minimize swapping, and (2) always give preference to the highest-priority process, even though that may mean swapping when swapping is not necessary. Suggest an intermediate policy that tries to balance the concerns of priority and performance.

- 3.6** Table 3.13 shows the process states for the VAX/VMS operating system.

- Can you provide a justification for the existence of so many distinct wait states?
- Why do the following states not have resident and swapped-out versions: Page Fault Wait, Collided Page Wait, Common Event Wait, Free Page Wait, and Resource Wait?
- Draw the state transition diagram and indicate the action or occurrence that causes each transition.

- 3.7** The VAX/VMS operating system makes use of four processor access modes to facilitate the protection and sharing of system resources among processes. The access mode determines

- Instruction execution privileges:** What instructions the processor may execute
- Memory access privileges:** Which locations in virtual memory the current instruction may access

Table 3.13 VAX/VMS Process States

Process State	Process Condition
Currently Executing	Running process.
Computable (resident)	Ready and resident in main memory.
Computable (outswapped)	Ready, but swapped out of main memory.
Page Fault Wait	Process has referenced a page not in main memory and must wait for the page to be read in.
Collided Page Wait	Process has referenced a shared page that is the cause of an existing page fault wait in another process, or a private page that is in the process of being read in or written out.
Common Event Wait	Waiting for shared event flag (event flags are single-bit interprocess signaling mechanisms).
Free Page Wait	Waiting for a free page in main memory to be added to the collection of pages in main memory devoted to this process (the working set of the process).
Hibernate Wait (resident)	Process puts itself in a wait state.
Hibernate Wait (outswapped)	Hibernating process is swapped out of main memory.
Local Event Wait (resident)	Process in main memory and waiting for local event flag (usually I/O completion).
Local Event Wait (outswapped)	Process in local event wait is swapped out of main memory.
Suspended Wait (resident)	Process is put into a wait state by another process.
Suspended Wait (outswapped)	Suspended process is swapped out of main memory.
Resource Wait	Process waiting for miscellaneous system resource.

The four modes are as follows:

- **Kernel:** Executes the kernel of the VMS operating system, which includes memory management, interrupt handling, and I/O operations
- **Executive:** Executes many of the OS service calls, including file and record (disk and tape) management routines
- **Supervisor:** Executes other OS services, such as responses to user commands
- **User:** Executes user programs, plus utilities such as compilers, editors, linkers, and debuggers

A process executing in a less privileged mode often needs to call a procedure that executes in a more privileged mode; for example, a user program requires an operating system service. This call is achieved by using a change-mode (CHM) instruction, which causes an interrupt that transfers control to a routine at the new access mode. A return is made by executing the REI (return from exception or interrupt) instruction.

- a. A number of operating systems have two modes, kernel and user. What are the advantages and disadvantages of providing four modes instead of two?
- b. Can you make a case for even more than four modes?

3.8 The VMS scheme discussed in the preceding problem is often referred to as a ring protection structure, as illustrated in Figure 3.18. Indeed, the simple kernel/user scheme, as described in Section 3.3, is a two-ring structure. A potential disadvantage of this protection structure is that it cannot readily be used to enforce a “need-to-know”

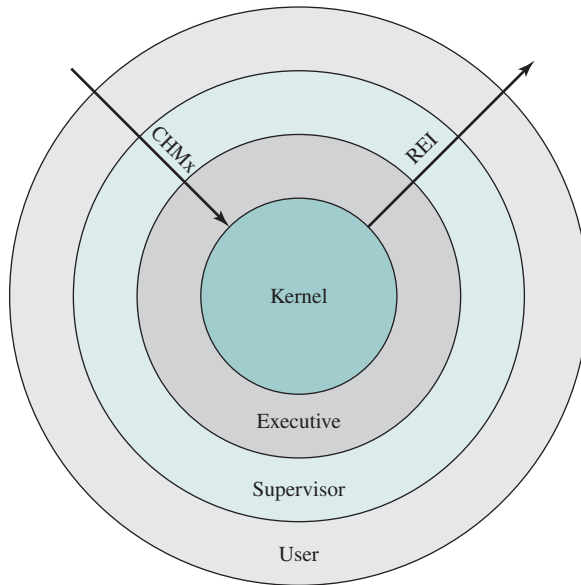


Figure 3.18 VAX/VMS Access Modes

principle. [SILB04] gives this example: If an object is accessible in domain D_j but not in domain D_i , then $j < i$. But this means that every object accessible in D_i is also accessible in D_j .

Explain clearly what the problem is that is referred to in the preceding paragraph.

3.9 Figure 3.8b suggests that a process can only be in one event queue at a time.

- a.** Is it possible that you would want to allow a process to wait on more than one event at the same time? Provide an example.
- b.** In that case, how would you modify the queueing structure of the figure to support this new feature?

3.10 In a number of early computers, an interrupt caused the register values to be stored in fixed locations associated with the given interrupt signal. Under what circumstances is this a practical technique? Explain why it is inconvenient in general.

3.11 In Section 3.4, it was stated that UNIX is unsuitable for real-time applications because a process executing in kernel mode may not be preempted. Elaborate.

3.12 You have executed the following C program:

```
main ()
{ int pid;
  pid = fork ();
  printf ("%d \n", pid);
}
```

What are the possible outputs, assuming the fork succeeded?